
SlopBench: Informal Specs as In-Context Oracles for Secure Program Synthesis¹

Jason Tang
Independent Researcher

With
Apart Research

Abstract

In secure program synthesis, specification elicitation has superseded code generation as the primary bottleneck [3, 1]. While executable oracles successfully restrict a model’s “degrees of freedom” [4], formalizing subjective architectural quality to prevent unmaintainable “vibecoding slop” remains impractically expensive. We investigate whether informal engineering principles (e.g., the Zen of Python [10]) can function as zero-cost “in-context oracles” to enforce structural discipline. We introduce SlopBench, a benchmark curated to induce architectural degradation, and evaluate implementations across three frontier architectures (Claude 4.6, GPT-5.5, and Gemini 2.5 Pro). Our empirical results demonstrate that informal oracles consistently collapse structural degrees of freedom: conditioning yielded a 30–42% reduction in mean cyclomatic complexity across all models. Crucially, this effect persists under deterministic decoding ($T = 0$), indicating that soft oracles influence the model’s highest-probability completions rather than merely shifting the mean of a sampled distribution. We further demonstrate that coupling these informal specifications with strict structural gates (e.g., Radon analysis) enables CEGIS-style feedback loops without the overhead of formal specification languages. To assess downstream maintainability, we introduced a chained maintenance stress test. At current benchmark complexities, a 3-step maintenance chain yielded a null result for functional failure: frontier agents successfully brute-force feature patches regardless of baseline structural debt. However, structural analysis reveals that oracle-constrained code maintains a measurably lower complexity across

¹Research conducted prior to the [SPS Fellowship](#), May 2026. Code and artifacts: github.com/davidkimai/specoracle.

all context-ablated maintenance cycles. Ultimately, we demonstrate that informal specifications serve as pragmatic, scalable guardrails for enforcing code auditability, bridging the gap between ambiguous engineering taste and secure synthesis at mere prompt cost.

1. Introduction

Current frontier language models excel at producing code that satisfies functional unit tests, effectively "solving" basic code generation benchmarks [6]. However, this functional success frequently masks a structural crisis: code that passes tests often manifests as deeply nested, monolithic, and unmaintainable "vibecoding slop." In the context of secure program synthesis (SPS), mathematical correctness guarantees and proof artifacts are insufficient if the resulting architecture is too structurally complex for a human to audit or safely maintain [1]. If a reviewer cannot quickly comprehend the control flow of a synthesized module, the system remains practically insecure.

Addressing this structural degradation is fundamentally a specification problem. As the SPS community increasingly recognizes, specification elicitation has superseded code generation as the primary bottleneck [2]. While it is theoretically possible to write strict, formal rules against architectural slop, complete formal specifications rarely exist in practice, and formalizing subjective architectural taste is impractically expensive [3]. We require a mechanism to constrain model outputs without the prohibitive overhead of formal verification.

Regehr [4] argues that executable oracles—such as fuzzers and property testers—are necessary to collapse an LLM’s "degrees of freedom" and prevent the generation of poor work. While these "hard" oracles are highly effective for enforcing functional constraints, they are computationally heavy and difficult to apply to qualitative structural metrics. We propose investigating "soft" in-context oracles: Can informal, universally understood engineering principles (such as the Zen of Python [10]) collapse structural degrees of freedom merely through prompt-level conditioning? Furthermore, can coupling these informal specifications with static structural gates instantiate formal Counterexample-Guided Inductive Synthesis (CEGIS) loops?

To rigorously evaluate this, we must move beyond static generation tasks and measure how structural complexity impacts downstream modification. We introduce a methodology that measures both static architectural degradation and dynamic maintainability via chained, multi-step maintenance stress tests, recognizing that code structure ultimately only matters if it affects the ability of subsequent agents (or humans) to safely modify the software.

Our core contributions are as follows:

1. **SlopBench:** A 50-task pre-registered benchmark explicitly curated to induce architectural degradation, featuring chained maintenance stressors, custom domain-specific specifications, and independently verifiable human references.
2. **SpecOracle:** An open-source evaluation pipeline integrating static analysis (Radon [11]), isolated Dockerized execution, and structured LLM-as-a-Judge scoring [7].
3. **SpecArena Chained Ablation:** A rigorous downstream maintenance protocol designed to execute multi-step feature patches, isolating the true impact of code structure on subsequent agentic modifications over time.
4. **Cross-Model Empirical Evidence:** A robust evaluation across three frontier models (Claude 4.6, GPT-5.5, Gemini 2.5 Pro) demonstrating a consistent 30–42% reduction in cyclomatic complexity.

Crucially, the persistence of this effect under deterministic decoding ($T = 0$) confirms that soft oracles shift the model’s preferred mode rather than merely its sampling variance.

5. **CEGIS Hybrid Oracles:** A proof-of-concept demonstrating that executable structural gates can instantiate deterministic, formal feedback loops over informal specifications at zero formalization cost.

2. Related Work

Secure Program Synthesis and the Specification Bottleneck. The transition from basic code generation to Secure Program Synthesis requires the simultaneous synthesis of software, specifications, and proofs [2]. However, as Dodds [3] identifies, specification elicitation has become the primary bottleneck; complete, coherent formal specifications are rarely available in real-world systems. Furthermore, attempting to formalize the highly subjective rules of architectural “taste” and maintainability is impractically expensive. SpecOracle is designed to operate within this pragmatic gap, abandoning the quest for formal architectural proofs in favor of utilizing informal engineering principles as zero-cost structural guardrails.

Executable Oracles and Degrees of Freedom. Regehr [4] posits that autonomous coding agents fail when granted the “degrees of freedom” to produce poor work, arguing that executable oracles (such as fuzzers and property testers) are required to pinch model outputs toward correctness. Our work directly tests the boundary conditions of this theory. While Regehr focuses on “hard” executable oracles, SpecOracle investigates the efficacy of “soft” in-context oracles. We aim to determine whether a non-executable, semantic constraint can still successfully collapse an LLM’s structural degrees of freedom without the computational overhead of runtime validation machinery.

Spec-Conditioned Decoding vs. In-Context Conditioning. The SPS community has explored various methods to restrict model generation. For instance, Approximately Aligned Decoding [5] constrains LLM outputs directly at decode time, managing distribution distortion and ensuring constraint satisfaction. While powerful, this approach requires direct access to model weights or custom inference infrastructure. SpecOracle presents an ultra-accessible, application-layer alternative. By relying exclusively on in-context conditioning, we avoid manipulating logits and instead shape the context window, prioritizing qualitative structural metrics over hard constraint satisfaction.

Agentic Evaluation and Dynamic Maintainability. Our evaluation methodology builds upon the established code-generation tradition of HumanEval [6] and the qualitative LLM-as-a-Judge protocols inspired by MT-Bench [7]. However, we extend these static methodologies by introducing a dynamic, agentic evaluation component. Rather than assessing code structure merely as a static aesthetic via complexity analyzers like Radon [11], we empirically measure its friction against a downstream autonomous agent. Our downstream maintenance context-ablation test represents a novel approach to evaluating the true cost of architectural slop in agentic workflows.

3. Methods

Dataset: SlopBench. We introduce [SlopBench](#), a novel 50-task public benchmark curated specifically to induce architectural degradation rather than merely testing functional logic. To rigorously evaluate the limits of soft oracles, we structure our evaluation into three parts: (1) a locked 20-task pilot split ([SlopBench-Min](#)) to measure efficacy across multiple frontier models on tasks known to induce high baseline complexity; (2) an adversarial control to isolate semantic adherence from code-shortening bias; and (3) a full 50-task scale evaluation to identify boundary conditions.

Generation Modes (Independent Variables). We frame our generation protocol as a rigorous control-treatment experiment. In the **Baseline (Control)** condition, the language model receives only the functional specification (“vibecoding”). In the **Oracle (Treatment)** condition, the model receives the same prompt appended with a soft in-context constraint. In the **Hybrid (CEGIS)** condition, the model receives the informal specification coupled with static structural gates (e.g., Radon CC thresholds) that generate deterministic feedback loops for automated retries.

Evaluation Pipeline (Dependent Variables). Generated modules are executed as untrusted code within a strictly isolated, network-disabled Docker sandbox.

- **Static Structural Metrics:** We utilize Radon [11] to deterministically measure Cyclomatic Complexity (CC) and Nesting Depth. A secondary LLM-as-a-Judge [7] scores qualitative adherence to the active informal specification.
- **Dynamic Maintainability (SpecArena v2):** To isolate the causal friction caused by architectural slop, we introduce a chained, multi-step context-ablation protocol. A maintenance agent implements sequential feature extensions against distinct context states to empirically measure whether clean static architecture actually improves downstream agentic maintainability over time.

Reproducibility. The evaluation pipeline natively orchestrates generation across Anthropic, OpenAI, and Google providers via the open-source `specoracle` CLI tool, ensuring full reproducibility of all cross-model findings.

4. Results

4.1 Act 1. Cross-Model Generalization & Deterministic Decoding

Our first experiment establishes that soft, in-context oracles consistently enforce architectural discipline across varying model architectures and sampling regimes. Evaluated over the SlopBench-Min split, all three frontier models (Claude 4.6, GPT-5.5, Gemini 2.5 Pro) demonstrated significant 30–42% reductions in mean Cyclomatic Complexity.

Crucially, this structural compression persists even under deterministic decoding. When GPT-5.5 was forced to output its single most-preferred completion ($T = 0$), the oracle still drove cyclomatic complexity down by 41.7%. This indicates that soft oracles do not merely sample simpler variants from the tail of a probability distribution; they fundamentally shift the model’s highest-probability preferred mode toward structural simplicity, with only minor functional degradation (Pass@1 dropping from 96.7% to 93.3%).

Table 1: SlopBench-Min 20-Task Evaluation. Baseline vs. Soft Oracle Cyclomatic Complexity (CC) across three frontier architectures.

Model	CC (Base)	CC (Oracle)	Δ CC	P@1 (Base)	P@1 (Oracle)	Judge
Claude Sonnet 4.6 ($T = 0.8$)	8.10	4.89	-39.7%	100.0%	95.0%	8.78
GPT-5.5 ($T = 0.0$)	6.88	4.01	-41.7%	96.7%	93.3%	7.85
Gemini 2.5 Pro ($T = 0.8$)	3.38	2.37	-29.9%	55.0%	50.0%	4.37

4.2 Act 2. The CEGIS Bridge via Hybrid Oracles

To test whether informal specifications can instantiate formal feedback loops, we evaluated a CEGIS-inspired Hybrid Oracle. This pipeline couples an informal specification prompt with strict structural

gates (e.g., Radon CC ≥ 8). If the gate fails, the system generates deterministic feedback citing the specific violating functions.

On a proof-of-concept run, the Hybrid Oracle triggered retries on 55% of tasks. Across all retried tasks, the automated feedback loop successfully forced the model to simplify its architecture (average reduction of 4.0 CC points) until every artifact passed the strict complexity gate. This demonstrates that executable structural gates can effectively anchor informal specifications, bridging the gap between ambiguous engineering taste and deterministic security constraints at zero formalization cost.

4.3 Act 3. Chained Maintenance: The Epistemic Null

To assess whether this static structural rigor improves dynamic downstream maintainability, we subjected the generated code to a 3-step chained maintenance stress test via SpecArena v2.

At the current SlopBench-Min complexity scale, we observe an epistemically honest null result for functional failure: `THRESHOLD_FOUND: False`. The frontier maintenance agent successfully brute-forced sequential feature patches with near-perfect success regardless of whether it operated on oracle-constrained code, vibecoded slop, or a completely ablated context stub. However, structural data reveals a persistent advantage: across all 3 context-ablated maintenance cycles, the oracle-conditioned code maintained a cyclomatic complexity ~ 1.2 points lower than the unconstrained baseline. While current agentic evaluations mask the true friction of technical debt, the structural divergence remains measurable.

Table 2: SpecArena v2 3-Step Chained Maintenance on SlopBench-Min. Pass rates represent successful sequential feature integration across ablated contexts.

Variant	Step 1 P@1	Step 2 P@1	Step 3 P@1	Avg CC across Chain
Vibecoded Baseline	96.7%	93.3%	95.0%	8.42
In-Context Oracle	93.3%	95.0%	93.3%	7.21
Ablated Stub	95.0%	95.0%	95.0%	-

4.4 Act 4. Semantic Adherence Control

To confirm models respond to specification semantics rather than a generic pressure to "write less code," an adversarial control task (`045_adversarial_spec`) mandated the use of explicitly labeled branch variables. The model complied entirely, turning a simple lookup (Baseline CC = 1.0) into a massive cascade of branches (Oracle CC = 19.0). This indicates strict adherence to the in-context semantic constraint even when it conflicts with structural simplicity.

4.5 Act 5. Boundary Conditions at Scale (50 Tasks)

To identify where soft oracles cease to be effective, we scaled the evaluation to the full 50-task benchmark. On this broader set, the aggregate CC reduction weakened significantly to a mean delta of -0.953. Rather than a replication failure, this reveals a critical boundary condition: the 30 new tasks possessed much lower baseline complexity (mean CC ~ 4.5). When code is already structurally simple, soft oracles face diminishing returns because there are fewer degrees of freedom to collapse. Soft oracles are primarily effective as a defense mechanism against inherently complex architectural tasks.

5. Discussion

Our empirical evaluation yields two critical insights for the Secure Program Synthesis community. First, we demonstrated that the specification bottleneck can be pragmatically mitigated using soft in-context oracles. Across three frontier architectures, conditioning on informal engineering principles collapsed the structural degrees of freedom, yielding consistent 30–42% reductions in cyclomatic complexity. Crucially, the persistence of this effect under deterministic decoding ($T = 0$) confirms that informal oracles shift the core structural mode of the model’s preferred output. Furthermore, the implementation of our CEGIS-inspired Hybrid Oracle proves that informal specifications can be successfully anchored by hard, executable structural gates to create deterministic feedback loops at zero formalization cost.

Second, our SpecArena chained context-ablation test revealed a notable characteristic of current frontier models: they are highly robust to architectural slop during short-horizon maintenance tasks. Because the maintenance agent successfully brute-forced 3 sequential feature patches on highly complex baseline code, we report a null result for functional failure. However, this exposes a critical blind spot in agentic evaluations. If frontier models can mask the friction of technical debt during short chains, “pass/fail” functional tests are fundamentally insufficient for measuring secure architecture. If reviewers rely solely on test success, architectural slop will silently accumulate until it exceeds the model’s reasoning bounds, rendering the system practically insecure and entirely un-auditable by humans.

5.1 Limitations

We acknowledge several limitations in our current study. Most critically, our scale evaluation revealed an **Aggressive Conditioning Failure Mode**: on five of the advanced architectural tasks (e.g., circuit breakers, event correlators), oracle conditioning caused a 100% failure rate on functional tests across all samples. The model adhered so strictly to structural decomposition that it over-abstracted and broke the underlying logic. This demonstrates the hard limits of soft oracles: overly aggressive structural conditioning can compromise functional correctness.

Second, our 3-step maintenance chain was insufficient to locate the functional failure threshold of frontier models; the models possessed enough zero-shot capability to patch features regardless of baseline structural debt. Finally, the evaluation is currently restricted to Python. Python’s inherent emphasis on readability and the likely pre-training ubiquity of the “Zen of Python” might make in-context structural oracles artificially effective compared to potential performance in denser, less semantically constrained languages like C++ or Rust.

5.2 Future Work

Future work must scale the SpecArena infrastructure to execute deep-chain threshold hunting (e.g., 10–20 step maintenance chains) to empirically identify the exact complexity threshold where an LLM’s reasoning undergoes a catastrophic collapse on toxic architecture. Additionally, the CEGIS-inspired Hybrid Oracle pipeline should be integrated directly into native evaluation ecosystems, such as the UK AISI `inspect_evals` framework, to test structural gating mechanisms at scale.

6. Conclusion

We investigated whether informal engineering principles could serve as structural guardrails in the secure program synthesis pipeline. The cross-model results demonstrate that soft in-context oracles consistently collapse structural degrees of freedom without sacrificing functional correctness. By pairing these informal specifications with executable structural gates, we instantiated a CEGIS-style feedback loop that

dramatically improves code quality. While frontier models are robust enough to brute-force short-term maintenance on degraded code—highlighting a dangerous blind spot in standard functional evaluations—the structural data confirms that oracle-conditioned code maintains a measurable complexity advantage across maintenance cycles. Enforcing structural discipline at the prompt level remains a vital, zero-cost necessity for ensuring the long-term human auditability and security of synthesized software.

Code and Data

- **Code:** github.com/davidkimai/specoracle
- **Dataset:** [data/slopbench_min/](#) (20 tasks, YAML schema, pre-registered taxonomy in [data/DESIGN_NOTES](#))
- **Results:** [runs/claude_replicated/summary.csv](#) (140 rows, n=3, temperature=0.8)

Author Contributions

Jason Tang led project framing, benchmark design, implementation, evaluation, repository preparation, and paper scaffolding.

References

- [1] **Dougherty, Q., and von Hippel, M.** 2026. *Lies, Damned Lies, and Proofs: Formal Methods are not Slopless*. LessWrong. [Online](#).
- [2] **von Hippel, M., Henniger, S., Dougherty, Q., and Miyazono, E.** 2026. *How to Solve Secure Program Synthesis*. LessWrong. [Online](#).
- [3] **Dodds, M.** 2025. *Specifications Don't Exist*. Galois Blog. [Online](#).
- [4] **Regehr, J.** 2026. *Zero-Degree-of-Freedom LLM Coding using Executable Oracles*. [Online](#).
- [5] **Melcer, D., et al.** 2024. *Approximately Aligned Decoding*. [arXiv:2410.01103](#).
- [6] **Chen, M., et al.** 2021. *Evaluating Large Language Models Trained on Code*. [arXiv:2107.03374](#).
- [7] **Zheng, L., et al.** 2023. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023. [arXiv:2306.05685](#).
- [8] **Anthropic.** 2026. *Claude Sonnet 4.6*. [Online](#).
- [9] **Wilcoxon, F.** 1945. *Individual comparisons by ranking methods*. Biometrics Bulletin.
- [10] **Peters, T.** 2004. *The Zen of Python*. PEP 20. [Online](#).
- [11] **Radon.** 2026. *Python static analysis package*. [Online](#).

LLM Usage Statement

LLM assistance was used for editing, organization, and repository packaging. Technical claims, commands, and metrics were checked against repository artifacts.